



Лекція1: Архітектура ROS2 та побудова модульної робототехнічної системи

КНУБА

Викладач: Маринський Костянтин Володимирович

КУРС «ПРОГРАМУВАННЯ ДИНАМІЧНИХ СИСТЕМ»

Київ - 2026

Мета лекції

Після цього заняття ви матимете чітке розуміння фундаментальних концепцій ROS2 і будете готові до практичної роботи з системою. Ці знання стануть основою для всього курсу.

01

Що таке ROS2

Розуміти призначення та роль ROS2 як середовища для побудови робототехнічних систем

03

Як програми спілкуються

Знати механізм обміну повідомленнями через топіки — Publisher/Subscriber модель

02

Що таке нода

Розуміти концепцію ноди як окремого модуля, що виконує конкретну функцію в системі

04

Архітектура системи

Бачити загальну архітектуру майбутньої модульної системи та розуміти важливість правильного середовища

Зміст лекції

Ця лекція закладе основи вашого розуміння ROS2, охоплюючи як теоретичні аспекти, так і практичні кроки для початку роботи.

01

Вступ до ROS2

Що таке ROS2 та його ключові переваги.

02

Складність роботів

Чому сучасні роботи потребують модульного підходу.

03

Концепція Ноди

Розбір базового будівельного блоку ROS2 — ноди.

04

Механізми спілкування

Як ноди взаємодіють через топіки.

05

Архітектура системи

Огляд майбутньої модульної робототехнічної архітектури.

06

Важливість інженерії

Значення порядку та версіонування у розробці.

07

Практичні завдання

Встановлення ROS2, Turtlesim та організація хмарного сховища.

Що таке динамічна система?

Динамічна система — це система, яка **змінюється з часом**. Ключове слово — змінюється.

Приклади з життя

- 🚗 Автомобіль, що їде — змінює швидкість і положення
- 🌡 Температура в кімнаті — змінюється з часом
- 🧑 Людина, яка йде — постійно змінює координати
- 📈 Курс валют — змінюється щосекунди

У всіх цих прикладах:



Формально (простими словами)

Динамічна система = Стан + Час + Правило зміни

Для робота:

- Стан робота — це швидкість, координати, кут повороту, дані сенсорів
- І цей стан змінюється постійно.

Робот — це класична динамічна система.





Що таке ROS2?

Уявіть команду будівельників, де кожен виконує свою роль: один кладе цеглу, другий проводить електрику, третій монтує вентиляцію, четвертий встановлює двері. Якщо кожен працює ізольовано й не узгоджує дії з іншими — будинок просто не зберуть.

🏗️ Аналогія з прорабом

ROS2 — це як **прораб** на будівництві. Він не виконує жодної фізичної роботи сам, але організовує взаємодію між усіма учасниками, забезпечуючи злагоджену роботу всієї системи.

💡 Простими словами

ROS2 — це фреймворк, що дозволяє **багатьом незалежним програмам** обмінюватися даними в реальному часі. Не одна монолітна програма, а безліч малих модулів, що «розмовляють» між собою через стандартизовані канали.

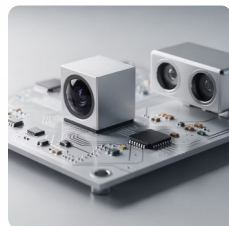
Чому робот — це складно?

Робот — це далеко не просто команда «поїхати вперед». Сучасна робототехнічна система є складним організмом із десятками підсистем, що працюють одночасно. Якщо спробувати написати всю цю логіку в одному файлі — він розросте до 20 000+ рядків, які неможливо підтримувати.



👁 Зір (камери)

Обробка відеопотоку, розпізнавання об'єктів та навігація у просторі



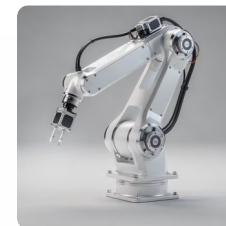
👂 Відчуття (сенсори)

Лідари, ультразвук, гіроскопи — сприйняття навколишнього середовища



🧠 Мозок (алгоритми)

Прийняття рішень, планування шляху, обробка даних у реальному часі



💪 Дії (двигуни)

Сервоприводи, колеса, маніпулятори — виконання фізичних команд

Рішення — модульність

У ROS2 кожен функціональний блок системи виноситься в окремий незалежний модуль — **модуль**. Це той самий принцип, за яким побудовані всі сучасні смартфони: кожна підсистема працює автономно і не заважає іншим.



Камера — окремий модуль

Відповідає виключно за захоплення та первинну обробку зображення. Не знає нічого про рух робота.



WiFi — окремий модуль

Забезпечує мережеву комунікацію незалежно від стану інших компонентів системи.



GPS — окремий модуль

Визначає координати й надає дані про місцезнаходження всім зацікавленим підсистемам.



Bluetooth — окремий модуль

Керує близькодіапазонним зв'язком, не втручаючись у роботу інших комунікаційних каналів.

📄 **Ключова перевага модульності:** якщо один модуль виходить із ладу — решта системи продовжує працювати. Це забезпечує відмовостійкість і спрощує налагодження.

Що таке Node (Нода)?

Нода — це окрема самостійна програма в екосистемі ROS2, яка виконує одну чітко визначену задачу. Всі ноди запускаються **паралельно** та можуть обмінюватися даними в реальному часі. Кожна нода може одночасно бути і відправником, і отримувачем повідомлень.



motion_node

Керує рухом робота: швидкість, напрямок, гальмування.
Отримує команди та перетворює їх на сигнали для двигунів.



vision_node

Обробляє відеопотік з камери: розпізнає об'єкти, визначає відстані, публікує результати аналізу.



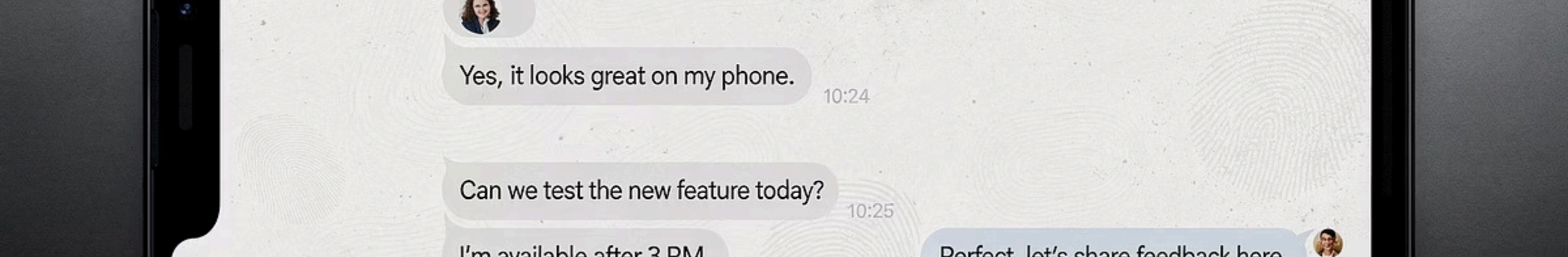
sensor_node

Зчитує показання датчиків (ультразвук, лідар, IMU) і публікує дані для інших нод системи.



gui_node

Відображає стан системи для оператора та транслює команди керування у відповідні топіки.



Як ноди спілкуються? Topic!

Основний механізм комунікації між нодами в ROS2 — це **Topic** (топик). Це асинхронний канал передачі даних, де одні ноди публікують повідомлення, а інші — підписуються на їх отримання. Найкраща аналогія — чат-група у Telegram.

Publisher (Видавець)

Нода, яка **надсилає** повідомлення у топик. Наприклад, `gui_node` публікує команди руху: «їдь вперед зі швидкістю 0.5 м/с» у топик `/cmd_vel`.



Topic (Топик)

Іменований канал зі строго визначеним **типом даних**. Наприклад, `/cmd_vel` передає повідомлення типу `geometry_msgs/Twist`.

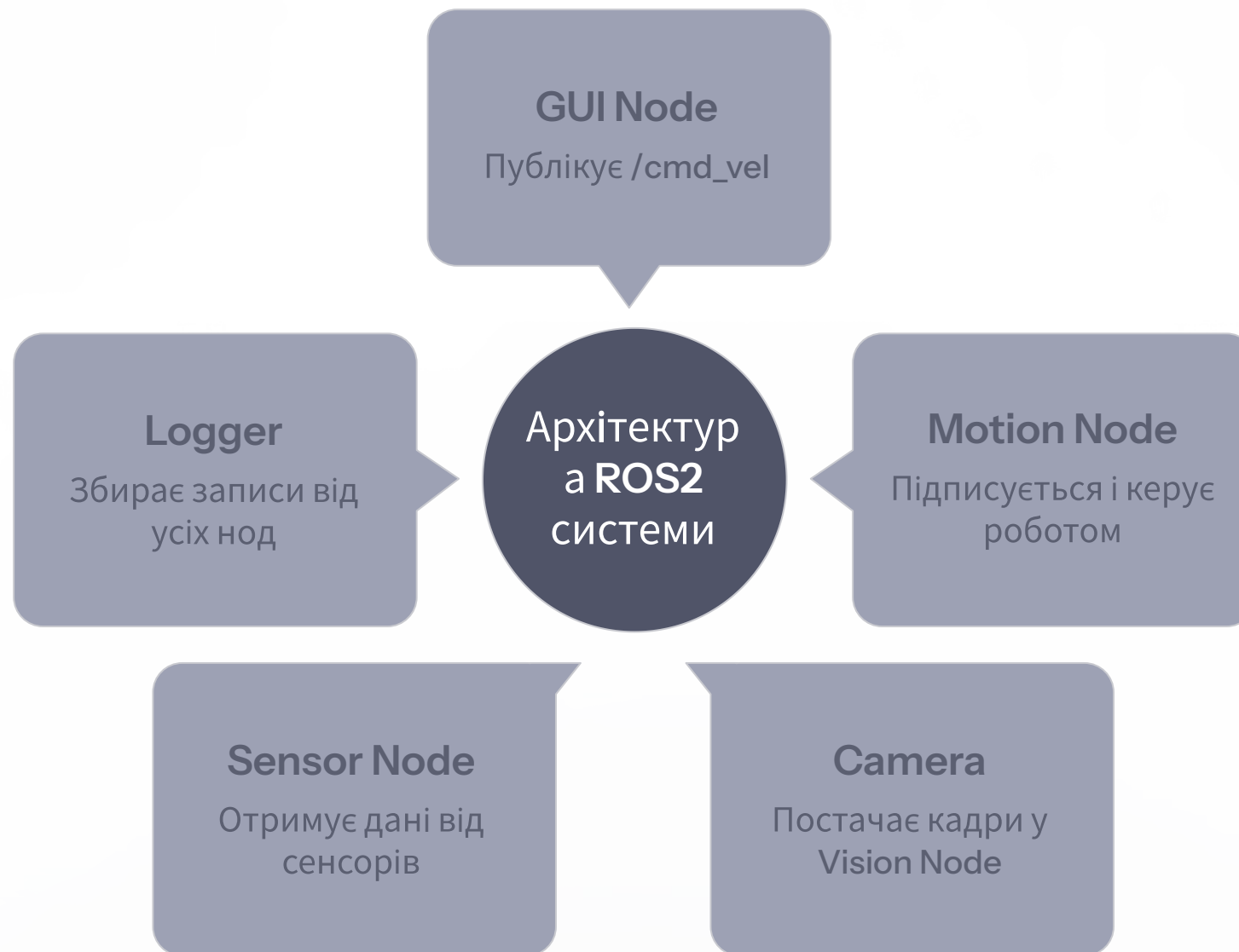


Subscriber (Підписник)

Нода, яка **отримує** повідомлення з топіку. Наприклад, `motion_node` підписується на `/cmd_vel` і перетворює команди на рух коліс.

Архітектура нашої системи

Протягом курсу ми будуватимемо повноцінну модульну систему, де кожна команда розробляє свою ноду. Ось як виглядає загальна архітектура взаємодії компонентів:



Ця архітектура демонструє ключову перевагу ROS2: кожна команда розробляє свою ноду незалежно, а система об'єднує їх через стандартизовані топіки. Зміна однієї ноди не зламає іншу — якщо інтерфейс (топик) залишається незмінним.

Принцип курсу: ми будуємо систему

Ми не просто пишемо код — ми проектуємо та будуємо **інженерну систему**. Це фундаментальна різниця між студентським проектом і продуктом, готовим до реального використання. Кожне рішення, яке ви приймаєте, впливає на надійність і підтримуваність усієї системи.

Кожна нода — завершений модуль

Нода повинна мати чітку відповідальність, коректно ініціалізуватись та завершуватись, обробляти помилки і бути покритою документацією.

Система має бути стабільною

Жодна нова функція не повинна порушувати роботу вже наявних. Перш ніж додати нове — переконайтесь, що старе працює коректно.

Завжди можна повернутись

Кожна значуща зміна фіксується. Якщо щось зламалося — у вас є робоча версія, до якої можна відкотитись за хвилину.

Порядок у файлах обов'язковий

Чітка структура директорій, зрозумілі назви файлів і модулів — це не бюрократія, а інструмент, що економить години пошуку помилок.

Чому порядок і версіонування — це частина інженерії

😬 Сценарій без версіонування

Ви будете машину. Хтось каже: «Я трохи змінив мотор» — і машина перестає заводитись. Попередньої версії немає. Що саме змінили — невідомо. Скільки часу знадобиться на відновлення? Дні? Тижні?

- Неможливо визначити причину поломки
- Втрачено весь прогрес
- Команда не може продовжити роботу

✅ Сценарій з версіонуванням

Та сама ситуація — але у вас є Git-репозиторій з чіткою історією змін. Ви бачите точно, що і коли змінювалось, і за кілька хвилин повертаєтесь до робочої версії.

- **Версіонування** — фіксація кожної значущої зміни
- **Структура** — зрозуміла організація коду та файлів
- **Backup** — резервні копії в хмарному сховищі

📋 💡 Версіонування — це не додаткова робота. Це страховка, що захищає тижні вашої праці від однієї необережної зміни.

Практичне завдання №1: Встановлення ROS2

Перший крок — налаштування єдиного робочого середовища для всієї команди. Це критично важливо: якщо у кожного учасника різна версія ROS2, спільна розробка стає неможливою.

1

Завантажити

Офіційний дистрибутив ROS2 **Humble Hawksbill** з [ros.org](https://www.ros.org) — LTS-версія з підтримкою до 2027 року

2

Встановити

Дотримуватись офіційної інструкції для вашої ОС. Перевірити всі залежності та змінні середовища

3

Перевірити

Виконати `ros2 --version` та переконатись, що версія збігається з версіями всіх членів команди

📌 ⚠️ Різні версії ROS2 (Humble vs Iron vs Jazzy) несумісні між собою так само, як різні «розетки й напруга» в різних країнах. Команда **обов'язково** використовує одну й ту саму версію.

Практичне завдання №2: Turtlesim — «Hello World» для роботів

Turtlesim — це вбудований симулятор ROS2, де черепашка рухається по екрану. Це найкращий перший приклад для розуміння того, як ноди взаємодіють через топіки.

turtlesim_node

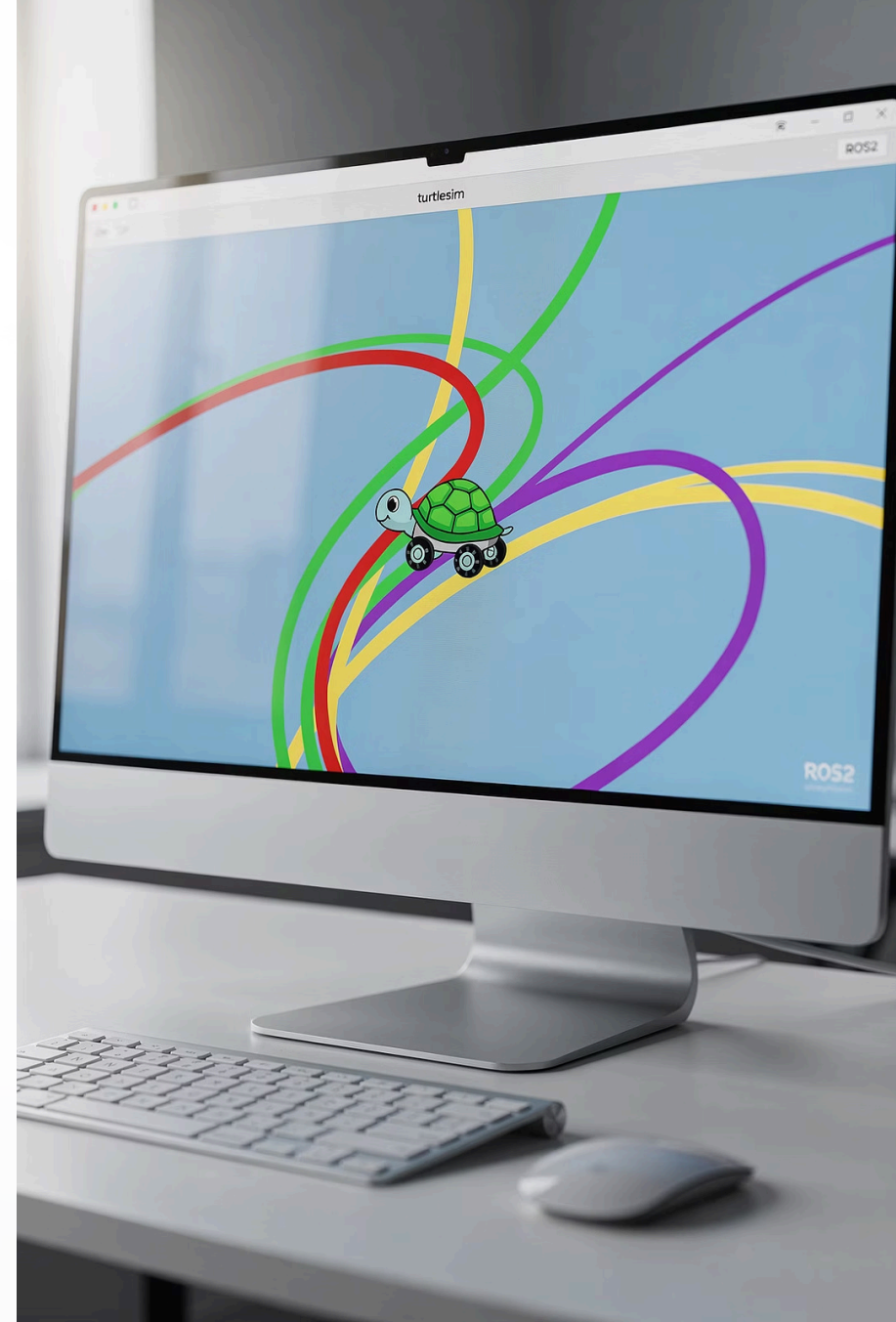
Нода-рендерер: відображає черепашку, підписана на топік `/turtle1/cmd_vel` і реагує на команди руху

turtle_teleop_key

Нода-контролер: зчитує натискання клавіш і публікує команди швидкості у той самий топік

`/turtle1/cmd_vel`

Топік між нодами: передає повідомлення типу `Twist` — лінійна та кутова швидкість черепашки



Практичне завдання №3: Організація хмарного сховища

Третє завдання — створити впорядковану структуру для збереження результатів роботи в OneDrive. Інженер — це не хаос, а порядок. Ваш код повинен бути захищений від будь-яких непередбачуваних ситуацій.

1 Збереження прогресу

Усі лабораторні роботи та проміжні результати зберігаються у структурованих папках. Якщо ноутбук зламається — ваша робота залишається в хмарі.

2 Захист від втрати коду

Регулярне завантаження змін до хмарного сховища гарантує, що жодна аварія з обладнанням не знищить тижні вашої праці.

3 Контроль версій та перевірка

Викладач перевіряє роботи через спільний OneDrive. Чітка структура папок і правильні назви файлів — обов'язкова умова прийняття роботи.

📌 Три кити надійної розробки: **версіонування + структура + backup**. Це не бюрократичні вимоги — це інструменти, що відрізняють інженера від аматора.